

Reviewer Pack — Hochster Betti Sum of Monotone f Lower-Bounds Log-Rank of IND...

Entry 06221fec8ebe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 10:04:12 UTC

Hochster Betti Sum of Monotone f Lower-Bounds Log-Rank of IND-Lifted Matrix

Entry ID: 06221fec8ebe **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 10:04:12 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial commutative algebra — specifically Hochster’s formula for the multigraded Betti numbers of the Stanley-Reisner ring $k[\Delta]$ of a simplicial complex via reduced simplicial homology of vertex-restrictions Δ_σ over F_2 (Hochster 1977; Bruns-Herzog Ch. 5); the total Hochster Betti sum $\beta(\Delta) := \sum\{\sigma \subseteq V\} \sum\{i \geq -1\} \dim_{F_2} \tilde{H}_i(\Delta_\sigma; F_2)$ is *poly-time computable from the face list and is essentially absent from communication-complexity and lifting literature*. **Field B** (complexity object): *Deterministic two-party communication complexity $D(f \circ \text{IND}_2^k)$ of Raz-McKenzie style lifted Boolean functions, accessed through the F_2 log-rank lower bound $D(M) \geq \log_2 \text{rank}_{F_2}(M)$; f is monotone on k variables and IND_2 is the 2-bit Index gadget (Alice holds 1 pointer per coordinate, Bob holds the 2-bit array), the canonical Göös-Pitassi-Watson / Raz-McKenzie lifting setup whose monotone-circuit and proof-complexity consequences are open above the certificate-complexity barrier.*

Statement:

Let $f: \{0,1\}^k \rightarrow \{0,1\}$ be monotone ($k \leq 6$), let $\Delta_f := \{S \subseteq [k] : f(1_S) = 0\}$ be its (downward-closed) Stanley-Reisner simplicial complex, and let $\beta(\Delta_f) := \sum\{\sigma \subseteq [k]\} \sum\{i = -1\}^{\lfloor |\sigma|-1 \rfloor} \dim_{F_2} \tilde{H}_i(\Delta_{\{f,\sigma\}}; F_2)$ be the total Hochster Betti sum, where $\Delta_{\{f,\sigma\}} = \{S \in \Delta_f : S \subseteq \sigma\}$. Then the F_2 log-rank of the IND_2 -lifted communication matrix satisfies $\log_2 \text{rank}_{F_2}(M_{\{f \circ \text{IND}_2^k\}}) \geq \lceil \log_2(1 + \beta(\Delta_f)) \rceil + 1$; a single monotone f for which $\log_2 \text{rank}_{F_2}(M_{\{f \circ \text{IND}_2^k\}}) < \lceil \log_2(1 + \beta(\Delta_f)) \rceil + 1$ refutes the conjecture.

Rationale (proposer’s reasoning):

Hochster’s formula expresses Betti numbers of $k[\Delta_f]$ as homological obstructions to contractibility of restrictions of Δ_f , which encode the hierarchy of minimal certificates of f at every coordinate-subset scale; lifting theorems pump exactly such ‘multiscale certificate structure’ from f into the communication matrix of $f \circ \text{IND}_2^k$ via the gadget. Because F_2 log-rank is an oblivious lower bound on D , and Hochster sums add up obstructions across all scales rather than just the worst minterm/maxterm (which is ordinary certificate complexity), the conjecture refines the standard certificate-lifting bound by the fine homological structure of the false-set complex. The bridge is genuinely new: Stanley-Reisner Betti theory has been used in algebraic statistics and toric geometry but, to our knowledge, never against IND-lifted matrices.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 77c02f8a008a85e7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 120 instances ($k \in \{3, 4, 5, 6\}$, 30 seeds each), the conjecture is SUPPORTED iff every instance satisfies $\log_2 \text{rank}_{\{F_2\}}(M_{\{f \cdot \text{IND}_2^k\}}) \geq \lceil \log_2(1 + \beta(\Delta_f)) \rceil + 1$ (support_fraction = 1.0) AND Spearman ρ between $\log_2(1 + \beta)$ and $\log_2 \text{rank} \geq 0.5$. Any single violating f falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.94	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Hochster formula Stanley-Reisner Betti numbers communication complexity lifting - log-rank F_2 lifted communication matrix Index gadget monotone Raz-McKenzie - simplicial homology Stanley-Reisner monotone Boolean function log-rank lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for j in range(n):
        i_max = -1
        for i in range(rank, m):
            if A[i][j] == 1:
                i_max = i
                break
        if i_max != -1:
            A[rank], A[i_max] = A[i_max], A[rank]
            for i in range(m):
                if i != rank and A[i][j] == 1:
```

```

        for k in range(n):
            A[i][k] ^= A[rank][k]
        rank += 1
    return rank

def nullity(A):
    return len(A) - gaussian_elimination(A)

def compute_boundary_matrix(C, i):
    m, n = len(C), len(C[0])
    B = [[C[j][i] for j in range(m)] for i in range(n)]
    return B

def compute_Homology_dimension(B_i, B_ip1):
    nullity_B_i = nullity(B_i)
    rank_B_ip1 = gaussian_elimination(B_ip1)
    return nullity_B_i - rank_B_ip1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    k_values = [3, 4, 5, 6]
    instances_tested = 0
    total_metric_value = 0.0
    support_count = 0

    for k in k_values:
        for _ in range(30):
            # Generate a random monotone function f
            antichain = [random.sample(range(k), i) for i in range(1, k)]
            f = lambda x: all(x[i] <= x[j] for i, j in combinations(range(k), 2))

            # Build  $\Delta_f$ 
            Delta_f = [S for S in range(1 << k) if all(f(bin(S)[2:].zfill(k)[:i+1]) == 0 for i in range(1, k))]

            # Compute  $\beta(\Delta_f)$ 
            beta_Delta_f = sum(sum(compute_Homology_dimension(compute_boundary_matrix(C, i), compute_boundary_matrix(C, i+1)) for i in range(1, k)) for _ in range(30))

            # Build the IND_2-lifted matrix  $M_{\{f \circ \text{IND}_2^k\}}$ 
            M = [[f(bin(i)[2:].zfill(k)[:j+1]) + bin(j)[2:].zfill(k)[:i+1]] for j in range(4**k)] for i in range(4**k)]

            # Compute  $\text{rank}_{\{F_2\}}(M)$ 
            rank_M = gaussian_elimination(M)

            # Check the inequality
            metric_value = max(rank_M, math.ceil(math.log2(1 + beta_Delta_f)) + 1)
            instances_tested += 1
            total_metric_value += metric_value

            if metric_value == math.ceil(math.log2(1 + beta_Delta_f)) + 1:
                support_count += 1

    mean_metric_value = total_metric_value / instances_tested
    support_fraction = support_count / instances_tested

    return {

```

```

        "metric_name": "log_2 rank_{F_2}(M_{f \circ IND_2^k})",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": support_fraction == 1.0,
        "counterexample": "" if support_fraction == 1.0 else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction == 1.0:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[0]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d932602.py", line 99, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d932602.py", line 63, in run_trial
    Δ_f = [S for S in range(1 << k) if all(f(bin(S)[2:].zfill(k)[:i+1]) == 0 for i in range(len(bin(S)[2:]
1))]
                                             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d932602.py", line 63, in <genexpr>
    Δ_f = [S for S in range(1 << k) if all(f(bin(S)[2:].zfill(k)[:i+1]) == 0 for i in range(len(bin(S)[2:]
1))]
                                             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d932602.py", line 60, in <lambda>
    f = lambda x: all(x[i] <= x[j] for i, j in combinations(range(k), 2))
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d932602.py", line 60, in <genexpr>
    f = lambda x: all(x[i] <= x[j] for i, j in combinations(range(k), 2))
                  ~~~~~
IndexError: string index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` before producing any data, due to a bug in constructing Δ_f (slicing a bit-string of variable length rather than the zero-padded representation). No instances were evaluated, so neither support fraction nor Spearman ρ could be computed. | next: Fix the Δ_f construction by iterating S over subsets of $[k]$ using bit-indexing on the zero-padded length- k string (or use bitmask membership directly), then rerun the 120-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	194988
2	preregistration	claude_max	opus	0	0	6101
3	novelty	claude_max	opus	0	0	4599
4	novelty	claude_max	opus	0	0	6582
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13388
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11992
7	judge	claude_max	opus	0	0	6225

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 243874 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/06221fec8ebe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/06221fec8ebe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/06221fec8ebe.tar.gz` (if generated)